NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 06

AIM :- Implement a denoising autoencoder for image restoration.

OBJECTIVE :-

1. To understand the concept of a denoising autoencoder.
2. To add noise to images and create corrupted training data.
3. To train an autoencoder to reconstruct clean images from noisy inputs.
4. To evaluate the quality of restored images.
5. To compare noisy, original, and reconstructed images.

THEORY :-

Denoising Autoencoder :-

A Denoising Autoencoder is a neural network trained to reconstruct a clean input image from its corrupted or noisy version. It consists of an Encoder, Latent Representation, and Decoder.

The Encoder compresses the noisy input into a lower-dimensional latent representation. The Decoder then uses this representation to reconstruct the original clean image.

Noise Addition :-

Gaussian noise can be added to the original image to create a noisy input:

$$X_noisy = X + \varepsilon$$

where X is the original image and ε represents the added noise. The pixel values are then clipped to the range 0 to 1.

Reconstruction Loss :-

The model compares the reconstructed image with the original clean image using Mean Squared Error (MSE):

$$L = (1/N)\ \Sigma\ (Xi - \hat{X}i)^2$$

PROCEDURE :-

1. Import TensorFlow and Matplotlib.
2. Load the MNIST dataset.
3. Normalize image pixel values between 0 and 1.
4. Add Gaussian noise to the training and testing images.
5. Clip noisy pixel values between 0 and 1.
6. Flatten the 28×28 images into 784-dimensional vectors.
7. Design the Encoder using Dense layers.
8. Create a lower-dimensional latent representation.
9. Design the Decoder to reconstruct the clean image.
10. Compile the model using Adam optimizer and MSE loss.
11. Train the model using noisy images as input and clean images as target.
12. Evaluate the model on noisy test images.
13. Plot training and validation loss.
14. Display noisy, original, and reconstructed images.

CODE :-

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt


(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()


x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


noise_factor = 0.4


x_train_noisy = x_train + noise_factor * np.random.normal(

    loc=0.0,

    scale=1.0,

    size=x_train.shape

)


x_test_noisy = x_test + noise_factor * np.random.normal(

    loc=0.0,

    scale=1.0,

    size=x_test.shape

)


x_train_noisy = np.clip(

    x_train_noisy,

    0.0,

    1.0

)


x_test_noisy = np.clip(

    x_test_noisy,

    0.0,

    1.0

)
```

```
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

x_train_noisy = x_train_noisy.reshape(-1, 784)
x_test_noisy = x_test_noisy.reshape(-1, 784)

input_img = tf.keras.Input(shape=(784,))

encoded = tf.keras.layers.Dense(
    128,
    activation="relu"
)(input_img)

encoded = tf.keras.layers.Dense(
    32,
    activation="relu"
)(encoded)

decoded = tf.keras.layers.Dense(
    128,
    activation="relu"
)(encoded)

decoded = tf.keras.layers.Dense(
    784,
    activation="sigmoid"
)(decoded)

autoencoder = tf.keras.Model(
    input_img,
    decoded
)

autoencoder.compile(
    optimizer="adam",
```

```
    loss="mse"
)

history = autoencoder.fit(
    x_train_noisy,
    x_train,
    epochs=10,
    batch_size=128,
    validation_split=0.2
)

test_loss = autoencoder.evaluate(
    x_test_noisy,
    x_test,
    verbose=0
)

print(
    "Test Reconstruction Loss:",
    test_loss
)

reconstructed = autoencoder.predict(
    x_test_noisy[:10],
    verbose=0
)

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
```

```
    label="Validation Loss"
)

plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.title("Denoising Autoencoder Training Loss")

plt.legend()
plt.grid(True)

plt.show()

plt.figure(figsize=(12, 6))

for i in range(5):

    plt.subplot(3, 5, i + 1)

    plt.imshow(
        x_test_noisy[i].reshape(28, 28),
        cmap="gray"
    )

    plt.axis("off")

    plt.subplot(3, 5, i + 6)

    plt.imshow(
        x_test[i].reshape(28, 28),
        cmap="gray"
    )

    plt.axis("off")

    plt.subplot(3, 5, i + 11)
```

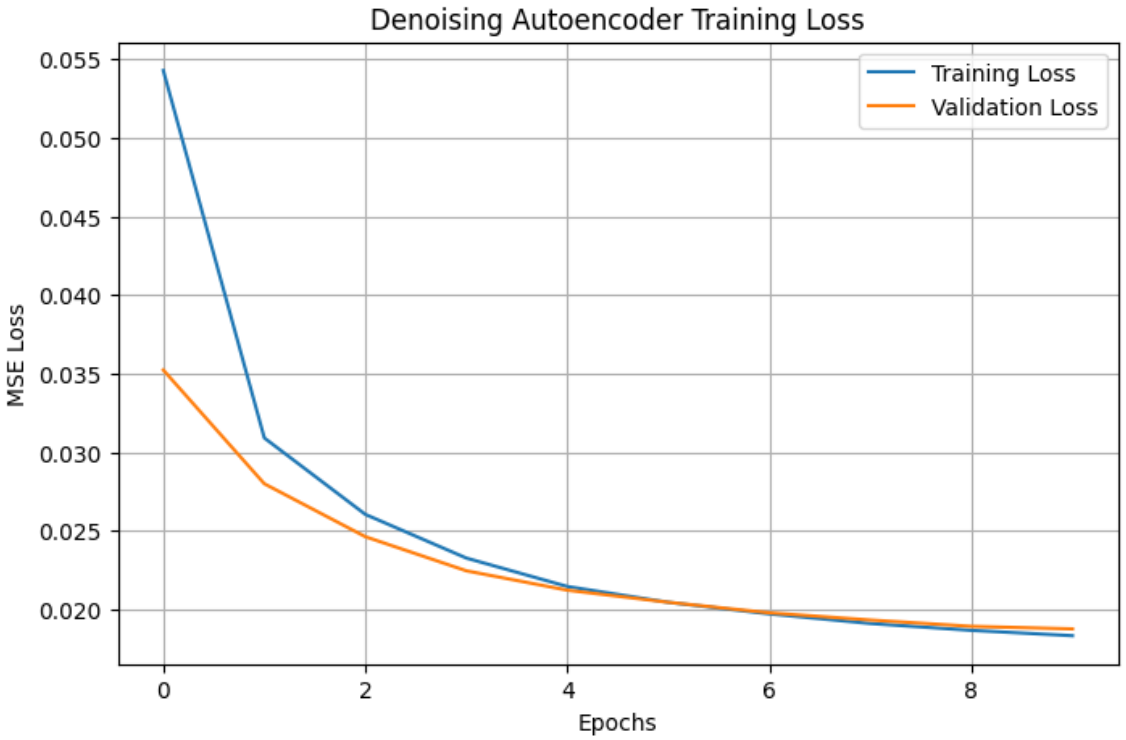
```
    plt.imshow(
        reconstructed[i].reshape(28, 28),
        cmap="gray"
    )

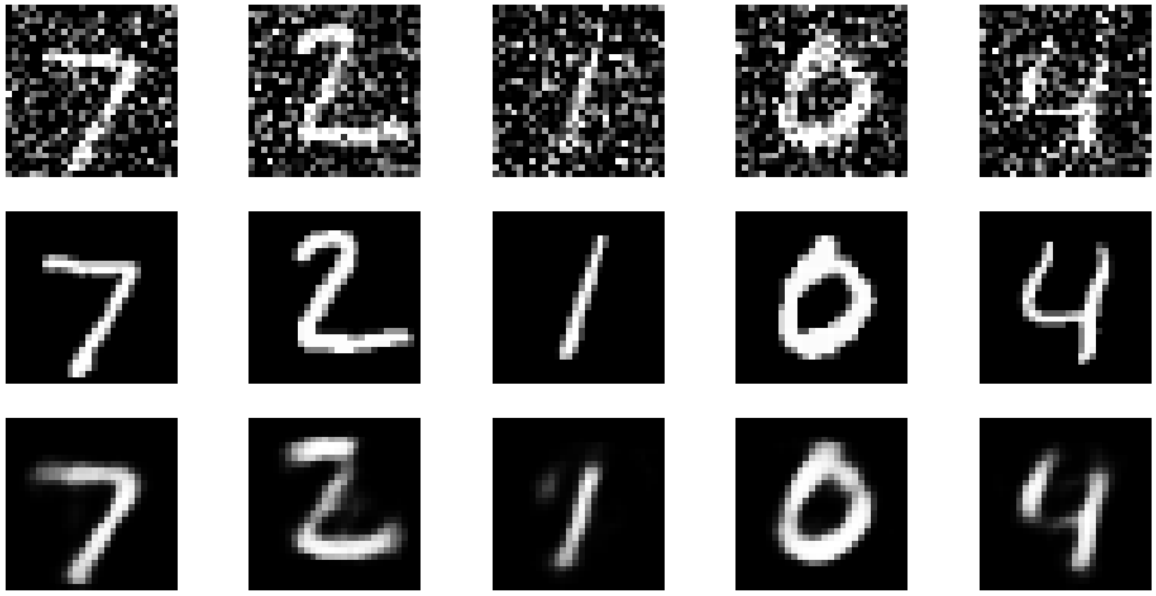
    plt.axis("off")

plt.show()
```

OUTPUT :-

```
Epoch 1/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - loss: 0.0543 - val_loss: 0.0352
Epoch 2/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.0309 - val_loss: 0.0280
Epoch 3/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.0260 - val_loss: 0.0246
Epoch 4/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0233 - val_loss: 0.0224
Epoch 5/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0214 - val_loss: 0.0212
Epoch 6/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 0.0204 - val_loss: 0.0204
Epoch 7/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 0.0197 - val_loss: 0.0198
Epoch 8/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0191 - val_loss: 0.0193
Epoch 9/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0187 - val_loss: 0.0189
Epoch 10/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.0183 - val_loss: 0.0188
Test Reconstruction Loss: 0.0182208064943552
```





CONCLUSION :-

The experiment successfully implemented a denoising autoencoder for image restoration. The model learned to remove noise from corrupted MNIST images and reconstruct images closer to the original clean images.